CPU Architecture

LAB3 assignment

Digital System Design with VHDL
(Simple RISC Multi-Cycle CPU design)

Hanan Ribo

תאריך- 18.05.2026

מגישים- נועה בן-שמואלי 322609090 ורועי שמש 322548165

המחלקה להנדסת חשמל

מטרת המעבדה

במעבדה זו נדרשנו לתכנן מערכת עיבוד מבוססת בקר- Controller-based processing machine בתצורת CPU. במעבדה תכננו מערכת המשלבת עקרונות לוגיקה מקבילית וסדרתית תוך שימוש בשיטות סימולציה מתקדמות. ובנוסף, תכננו בקר (Controller) המבוסס על יחידת בקרה- Control Unit, ויחידת נתיב נתונים- Datapath.
בביצוע המעבדה נדרשה הבנה עמוקה של ולידציה ווריפיקציה פונקציונלית של תכנון ארכיטקטורה.

הגדרת המערכת

המערכת שתוכננה היא מעבד רב-מחזורי (Multi-cycle CPU) בעל נתיב נתונים יחיד (BUS-1).
מטרתו היא להריץ קוד תוכנה נתון.

תיאור הארכיטקטורה-

המערכת מבוססת על הפרדה בין יחידת הבקרה ליחידת נתיב הנתונים-

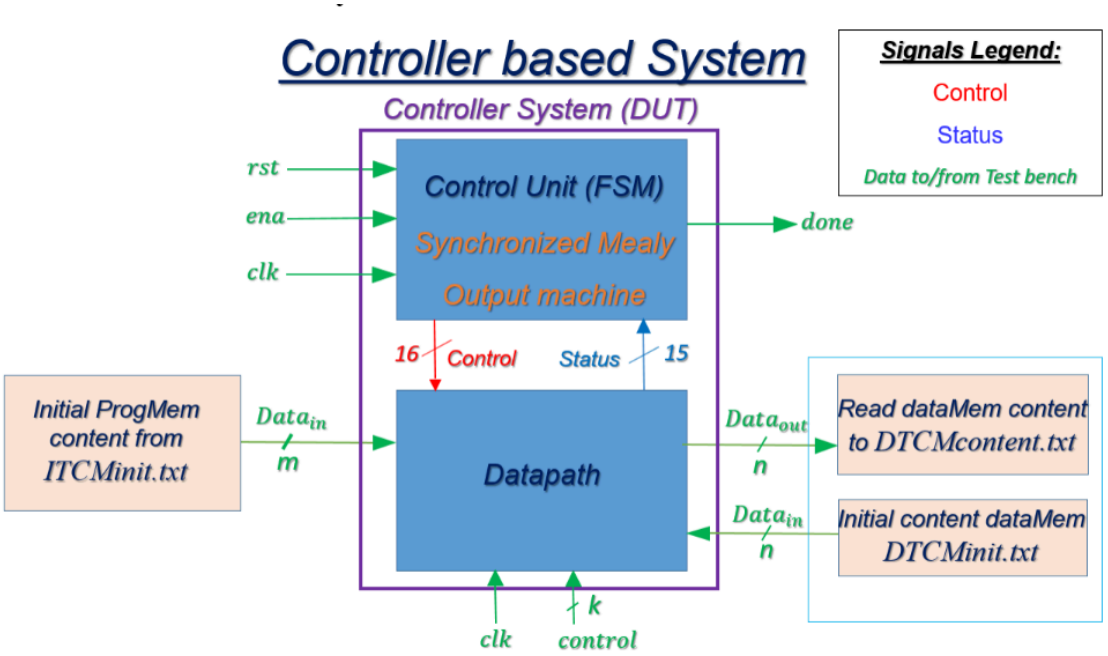
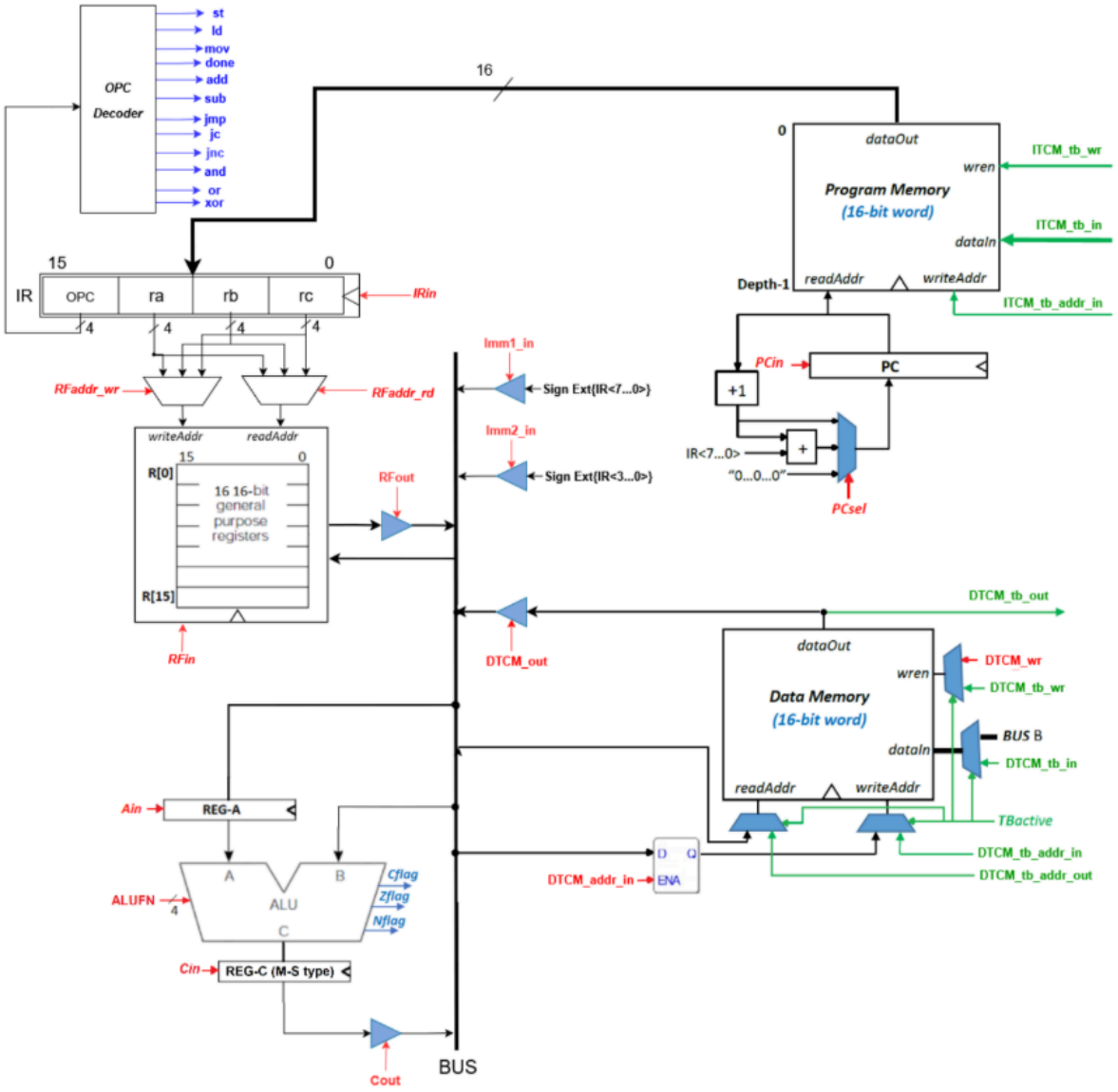


Figure 1: Overall DUT structure

1. **יחידת הבקרה (Control Unit):** ממומשת כמכונת מצבים מסוג מילי סינכרונית (Synchronized Mealy FSM). היא מקבלת אותות בקרה (`clk`, `rst`, `ena`), דגלי סטטוס (`Nflag`, `Zflag`, `Cflag`), ומפיקה את כל אותות הבקרה הנדרשים לנתיב הנתונים.
2. **יחידת נתיב הנתונים (Datapath):** מורכבת מזיכרון תוכניות (Program Memory/ITCM), זיכרון נתונים (Data Memory/DTCM), קובץ אוגרים (RF - Register File), אוגרים פנימיים (PC, IR, REG-A, REG-C), יחידה אריתמטית לוגית (ALU), ורכיבים נוספים המחוברים באמצעות אפיק נתונים יחיד (BUS).

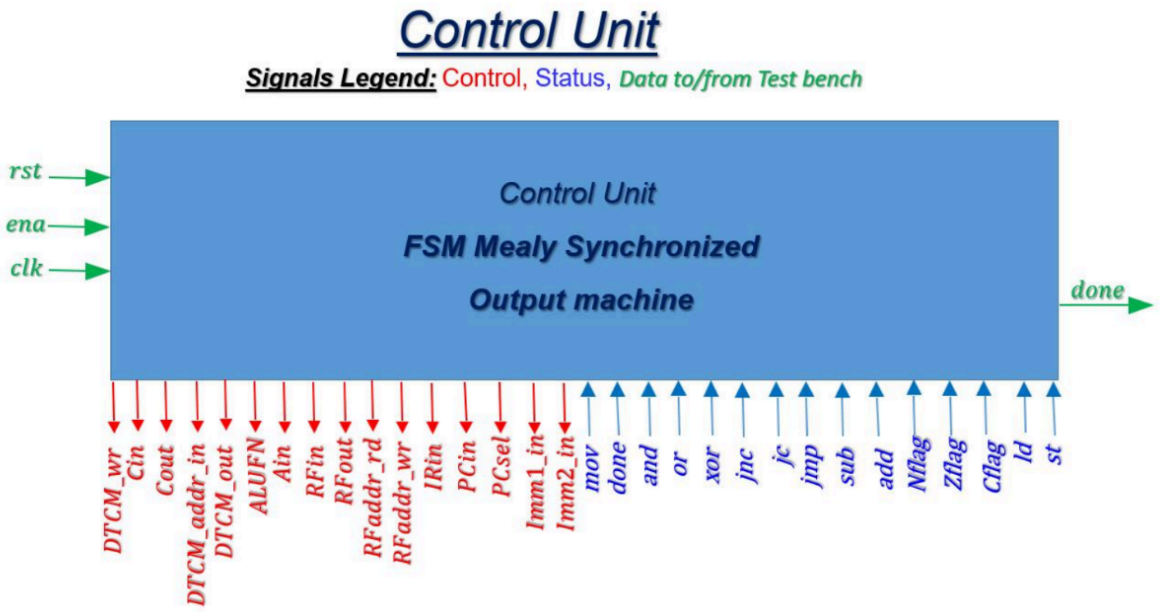


כניסות ויציאות המערכת מתחלקות לאותות בקרה, סטטוס ואותות נתונים המועברים אל ומאת סביבת הבדיקה (Test bench).

- **אותות בקרה ראשיים:** `clk` (שעון), `rst` (איפוס), `ena` (אפשור מערכת).
- **אותות נתונים ובקרה מה-TB ל-DUT:**
  - אותות אתחול זיכרון תוכניות ITCM להלן- `ITCM_tb_wr`, `ITCM_tb_addr_in`, `ITCM_tb_in`.
  - אותות אתחול זיכרון נתונים DTCM להלן- `DTCM_tb_wr`, `DTCM_tb_addr_in`, `DTCM_tb_in`.
- **אותות סטטוס וסיום (Outputs to TB):**

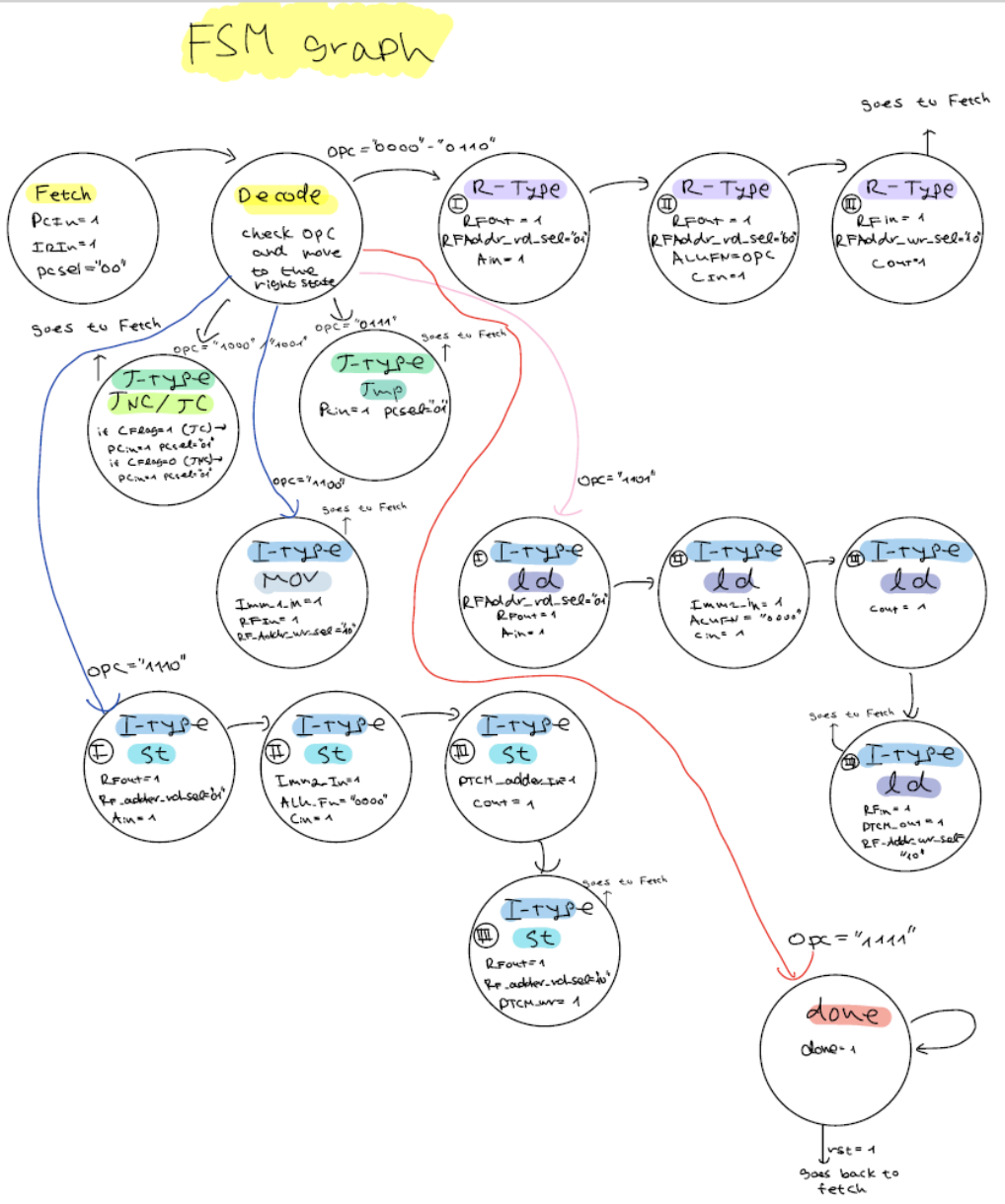
- `done`: אות מוצא המאותת ל-TB לקרוא את תוכן זיכרון הנתונים (DTCM) לאחר סיום ריצת הקוד.
- אותות קריאה מזיכרון נתונים: `DTCM_tb_out` (תוכן), `DTCM_tb_addr_out` (כתובת).

- **אותות פנימיים:** אותות בקרה (Control) מועברים מהבקר לנתיב הנתונים (16 קווים), ואותות סטטוס (Status) מועברים מנתיב הנתונים לבקר (15 קווים, כולל דגלי `Cflag`, `Zflag`, `Nflag`)



דרישות המימוש (Requirements)-

על מנת לממש את המערכת במשימה, כתבנו מערכת מצבים מפורטת המצורפת כקובץ נפרד להלן-



מכונת המצבים מבוססת על הארכיטקטורה הבאה-

1. מצב FETCH קבוע לטעינת ההוראה.
2. מצב DECODE של מחזור יחיד כדי למנוע שימוש באופקוד ישן מיד לאחר Reset.
3. פיצול למסלולים ייעודיים לפי סוג הפקודה (Opcodes).

1. <u>נקודת ההתחלה: FETCH ו-DECODE</u>

- Fetch: נקודת המוצא של כל פקודה במעבד. במצב זה, האותות 'PCin='1 ו-'IRin='1 עולים כדי לקדם את ה-PC ולנעול את הפקודה הבאה מהזיכרון אל תוך ה-IR. ברירת המחדל של בחירת ה-PC היא קידום רגיל (PCsel=00).

- Decode: מחזור שעון יחיד המוקדש לפענוח. כפי שרשום בתרשים וכפי שמופיע בקוד, במצב זה כל אותות הבקרה למסלול הנתונים נשארים כבויים כדי לאפשר לאופקוד החדש להתייצב במפענח. ממצב זה, ה-FSM בודק את קווי הסטטוס ומחליט לאיזה נתיב לפנות.

2. <u>מסלול פקודות חישוב: R-Type- add, sub, andop, orop, xorop</u>

מסלול זה מורכב מ-3 מצבים עוקבים הממומשים בקוד כ-S_R2 ,S_R1, ו-S_R3:

- מצב (S_R1): קריאת האופרנד הראשון. האותות RFout=1 ו-Ain=1 מופעלים. כתובת הקריאה נבחרת לפי שדה ה-8 ביטי של ההוראה RFaddr_rd_sel=01 שהם ביטים של אוגר rb והתוכן ננעל באוגר המבוא של ה-ALU.
- מצב (S_R2): קריאת אופרנד שני וביצוע. המעבד מוציא את האוגר השני RFaddr_rd_sel=00 שהוא אוגר rc אל הבאס (RFout=1) ומפעיל את Cin=1 כדי לנעול את תוצאת החישוב של ה-ALU באוגר C. קוד הפעולה הספציפי מוכתב ישירות על ידי האופקוד באמצעות האות המקבילי ALUFN <= opcode.
- מצב (S_R3): כתיבה חזרה. התוצאה מאוגר C מוזרמת לבאז (Cout=1) ונכתבת לתוך אוגר היעד ב- RFin=1 :Register File, כאשר כתובת אוגר היעד נבחרת לפי RFaddr_wr_sel=10 (אוגר ra).
- סיום: המצב הבא מוגדר כ-S_FETCH.

3. <u>מסלול פקודות זיכרון נתונים: I-Type- Load & Store</u>

א. פקודת טעינה מהזיכרון (ld)

הפקודה דורשת 4 שלבים (S_LD1 עד S_LD4) כדי לחשב כתובת, לגשת ל-DTCM ולשמור את הערך:

- מצב I ו-II: חישוב הכתובת. המעבד טוען את אוגר הבסיס ALU ל- RFaddr_rd_sel=01 , RFout='1', Ain='1' , ולאחר מכן מחבר אליו ערך מיידי קצר (Imm2_in='1', Cin=1) כאשר ה-ALUFN מקובע על פעולת חיבור ("0000").
- מצב III: הוצאת הכתובת המחושבת מאוגר C אל קווי הכתובת של הזיכרון (Cout=1).
- מצב IV: כתיבת הנתון מהזיכרון לאוגר. זיכרון הנתונים מוציא את התוכן לבאס (DTCM_out=1) והוא ננעל באוגר היעד ב- Register File RFaddr_wr_sel=10 ,RFin='1'. לאחר מכן חוזרים ל-FETCH.

ב. פקודת שמירה בזיכרון (st)

גם היא פקודה בת 4 שלבים (S_ST1 עד S_ST4):

- מצב I ו-II: חישוב הכתובת בדומה ל-LD (שימוש ב-Ain, חיבור ערך מיידי קצר ב-ALU ונעילה ב-Cin).
- מצב III: נעילת הכתובת. התוצאה מאוגר C מוציאה את הכתובת המחושבת, והאות DTCM_addr_in=1 מופעל כדי לנעול אותה באוגר הכתובות של הזיכרון.
- מצב IV: כתיבת המידע. האוגר שמכיל את המידע לשמירה מוציא את תוכנו לבאס המרכזי RFaddr_rd_sel=10 ,RFout='1' שהוא אוגר היעד ra שנקרא כעת כמקור , ואות הכתיבה

של הזיכרון מופעל (DTCM_wr=1) כדי לבצע את השמירה הפיזית ב-DTCM. בסיום, חוזרים ל-FETCH.

4. מסלול פקודת העברה: I-Type- mov

- S_MOV: זוהי פקודה מהירה בת מחזור אחד בלבד לאחר שלב הפענוח. היא עוקפת את ה-ALU לחלוטין: האות Imm1_in=1 מזרים את הקבוע ה-8-ביטי (המורחב) ישירות מה-IR לבאס, והאות RFin=1 נועל אותו מיד באוגר היעד (RFaddr_wr_sel=10). בסיום, המצב הבא הוא ישירות FETCH.

5. מסלול פקודות בקרת זרימה: J-Type- Jumps & Branches

א. קפיצה לא מותנית (jmp)

- S_JMP: המעבד מעדכן את ה-PC ישירות במחזור זה. האות PCin=1 מופעל, והבורר משתנה ל-PCsel=01. בחירה זו מורה לרכיב ה-PC_Mux לחשב את הכתובת החדשה לפי כתובת ה-PC הנוכחית בתוספת היסט (Offset) מורחב סימן שנלקח מה-IR. בסיום, חוזרים ל-FETCH.

ב. קפיצה מותנית (jc / jnc)

- S_JC / S_JNC: המצבים האלו מבצעים בדיקה לוגית של דגל הנשא (Cflag) שנשמר במחזור הקודם:
  - ב-S_JC: אם Cflag = '1', התנאי מתקיים והמעבד מפעיל את 'PCin='1 ו-"PCsel="01 כדי לבצע את הקפיצה. אם הדגל הוא '0', המעבד מתעלם וה-PC מקודם כרגיל.
  - ב-S_JNC: הפעולה הפוכה – הקפיצה מתבצעת רק אם Cflag = '0'.
  - שני המצבים מחזירים את המכונה מיד לאחר מכן ל-S_FETCH.

6. מצב סיום עצירה: Special- done

- S_DONE: פקודה מיוחדת המסמנת את סוף התוכנית. המערכת מפעילה את האות done_out=1 כדי לתת חיווי חיצוני לטסטבנץ'. כפי שניתן לראות היטב בלופ העצמי בתרשים ובקוד, המצב הבא של המצב הזה הוא הוא עצמו (next_state <= S_DONE). המעבד יישאר תקוע בלולאה זו ("Trap") ולא ימשיך לבצע פקודות עד אשר יתקבל אות איפוס חיצוני (rst=1) שיחזיר את המערכת ל-FETCH.

<u>ניתוח ואימות של התוצאות (Test and Timing Analysis)</u>

האימות הפונקציונלי של המערכת מתבצע באמצעות סימולציה מבוססת קבצים- File-based simulation. הסימולציה משתמשת בקבצי קלט ופלט כדי לבדוק את תקינות פעולת ה-CPU תחת תוכנית נתונה.

מטרת סביבת הבדיקה היא לאמת את תקינות תכנון ה-CPU (ה-DUT) על ידי הרצת קוד והשוואת תוצאות הריצה לתוצאות צפויות.

קבצי אתחול ופלט: סביבת הסימולציה משתמשת בשלושה קבצים עיקריים:
ITCMinit.txt - מכיל את תוכן זיכרון התוכנית ההתחלתי.
DTCMinit.txt - מכיל את תוכן זיכרון הנתונים ההתחלתי.
DTCMcontent.txt - קובץ פלט המכיל את תוכן זיכרון הנתונים לאחר סיום ריצת הקוד.

שימוש בכלי איכות תוכנה (SW QA): ההוריפיקציה מבוצעת על ידי הרצת יישומי בדיקה מוגדרים (Code Example 1 עד 6). כל יישום כולל חמישה קבצים, כולל קבצים בינאריים לאתחול זיכרונות (ITCM ו-DTCM) וקובץ של תוכן DTCM הצפוי לאחר ריצת הקוד.

תיאור ה- TB TOP: ה-Testbench הכללי (overall DUT) משמש לטעינת הקבצים ההתחלתיים לזיכרונות, הרצת מחזורי השעון עד לקבלת אות done, ובסיום - לקריאת תוכן זיכרון הנתונים וייצואו לקובץ DTCMcontent.txt לצורך השוואה למודל הזהב.

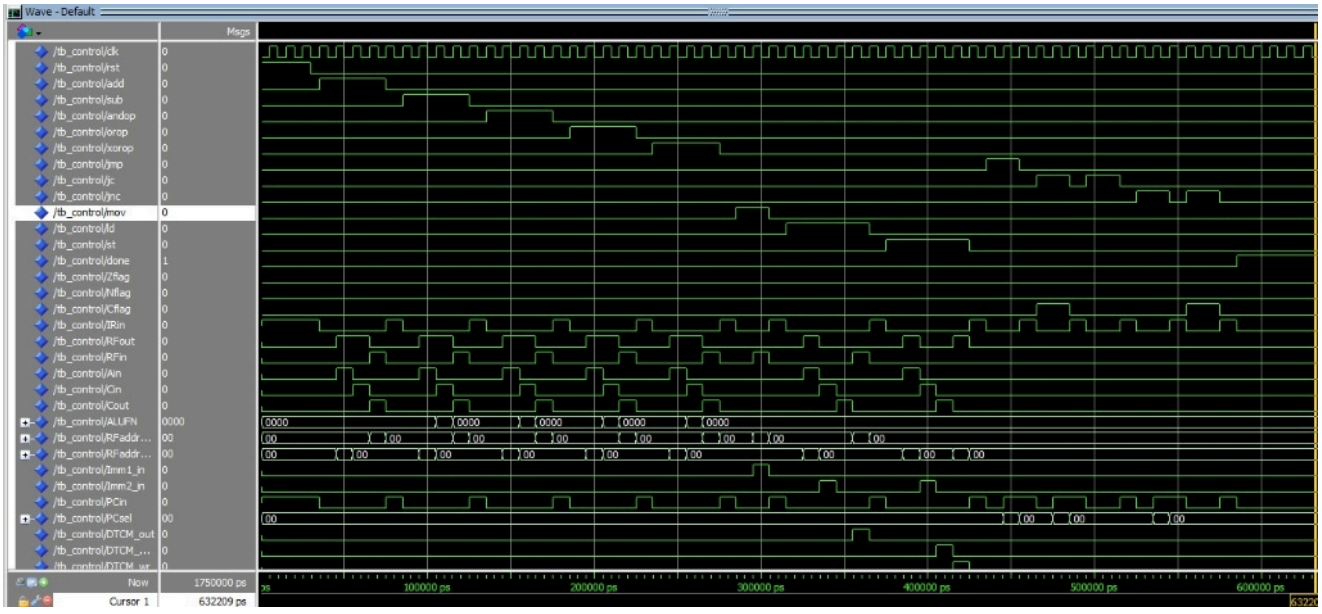
הוריפיקציה של המערכת בוצעה מול מודל זהב (Golden Model) תוך שימוש בשלושה תרחישי בדיקה (Testbenches) המדגימים התנהגויות שונות של המערכת:

הנה סיכום מלא ומאורגן של תוצאות סימולציית ה-Control Unit בעברית, המשלב את ניתוח הגלים עם אבני הדרך הכרונולוגיות (Callouts) של הפרויקט:

<u>פירוט נקודות בתרשים ה-wave</u>

1. יחידת ה- Control Unit:

תוצאות סימולציית tb_Control-



סיכום הסימולציה-

על פי תרשים הגלים וה-Testbench, סימולציית ה-**Control Unit** ניתן להבחין כי תכנון מכונת המצבים (FSM) **תקין לחלוטין** ומציג התנהגות צפויה-

- **מחזוריות ה-FETCH:** האות IRin עולה בדופק מחזורי וקבוע לאורך כל הריצה. כל רצף פעולות מסתיים בחזרה למצב ה-FETCH, כמצופה ממעבד סדרתי.
- **ביצוע פקודות:** המעבד מצליח לעבור בצורה נכונה בין פקודות חישוב (R-Type), פקודות ערך מיידי (MOV), גישות לזיכרון הנתונים (LD/ST) וניהול זרימת התוכנית (Jumps).
- **סיום:** עם קבלת פקודת הסיום, ה-FSM ננעל במצב העצירה ומספק אינדיקציה יציבה לטסטבנץ'.

ניתוח הסימולציה לפי נקודות זמן-

1. <u>שחרור מאיפוס וכניסה ל-FETCH</u> (סביב 30,000ps)-

האות rst יורד ל-'0' (Falling edge). מכונת המצבים משתחררת ממצב האיפוס ונכנסת אל המצב הראשוני S_FETCH.

2. <u>מחזור FETCH ראשון</u> (סביב 30,000ps - 40,000ps)

האותות IRin ו-PCin עולים יחד ל-'1'. בשלב זה פקודת ה-ADD נטענת מזיכרון התוכנית (ITCM) אל תוך אוגר ההוראות (IR), וערכו של ה-PC מקודם לקראת הפקודה הבאה.

3. <u>שלב ראשון בפקודות R-Type</u> (סביב 50,000ps)

ה-FSM עובר למצב S_R1 (כחלק מבדיקת פקודת ה-ADD הראשונה). האותות RFout ו-Ain עולים ל-'1'. האופרנד הראשון (rb) נקרא מתוך ה-Register File ונטען לאוגר הכניסה של ה-ALU A_reg.

4. <u>ביצוע פעולת חישוב ב-ALU</u> (סביב 100,000ps)

במהלך בדיקת פקודת החיסור (SUB), ה-FSM נמצא במצב S_R2. האות ALUFN משתנה לערך "0001" (opcode של חיסור) כדי להורות ל-ALU לבצע A - B.

5. <u>פקודת העברת קבוע לרגיסטר - MOV</u> (סביב 280,000ps)

המכונה מזהה פקודת mov ועוברת למצב S_MOV. האות Imm1_in עולה כדי לטעון את הקבוע ה-8-ביטי אל ה-BUS המרכזי , ובמקביל האות RFin מופעל כדי לכתוב את הערך הזה ישירות לאוגר היעד ב-RF במחזור יחיד.

6. <u>כתיבה לאוגר בפקודת טעינה - LD</u> (סביב 350,000ps)

מכונת המצבים מגיעה למצב האחרון של פקודת הטעינה (S_LD4). האות DTCM_out עולה ל-'1' וגורם לזיכרון הנתונים להזרים את המידע ל-BUS , בזמן ש-RFin דואג לשמירת המידע באוגר היעד.

7. <u>כתיבה לזיכרון בפקודת שמירה - ST</u> (סביב 420,000ps)

ה-FSM מגיע למצב הסופי של פקודת השמירה (S_ST4). האות DTCM_wr עולה ל-'1' (אות כתיבה אקטיבי) , והתוכן של האוגר נכתב בפועל אל הכתובת המבוקשת בתוך זיכרון הנתונים (DTCM).

8. ביצוע קפיצה לא מותנית - JMP (סביב 440,000ps)

ה-FSM נכנס למצב S_JMP. האות PCsel משתנה מהערך הדיפולטיבי "00" לערך "01". שינוי זה מנתב את ה-PC לקבל את כתובת היעד החדשה (PC + 1 + offset) במקום קידום רגיל.

9. קפיצה מותנית ממומשת - JC Taken (סביב 470,000ps)

פקודת קפיצה לפי עליית דגל נשא (jc) מופעלת כאשר דגל הנשא דלוק- ('Cflag = '1'). מאחר שהתנאי מתקיים, האות PCin עולה ל-'1' יחד עם "PCsel = "01 כדי לבצע את הסטת ה-PC אל עבר היעד.

10. קפיצה מותנית שלא ממומשת - JNC Not Taken (סביב 560,000ps)

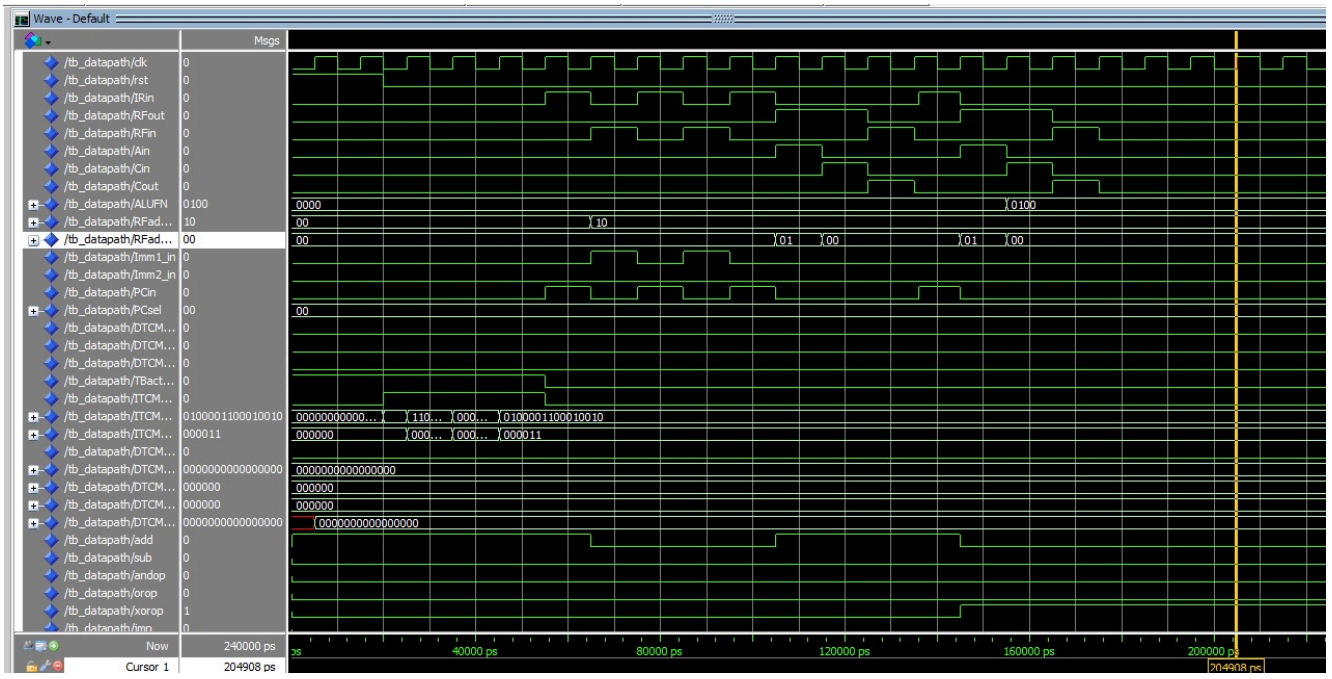
מבוצעת בדיקת פקודת jnc (קפיצה אם אין נשא) אך במצב שבו דגל הנשא דלוק בפועל- ('Cflag = '1'). כיוון שהתנאי נכשל, המעבד אינו מבצע קפיצה, האות PCin נשאר נמוך ('0') וה-PC אינו מתעדכן בכתובת המטרה.

11. הגעה למצב סיום התוכנית - S_DONE (סביב 590,000ps)

ה-FSM מגיע למצב העצירה הסופי S_DONE. האות done_out עולה ל-'1' קבוע ומסמן לטסטבנץ' שהתוכנית הסתיימה בהצלחה. המכונה נשארת במצב זה באופן קבוע- (next_state <= S_DONE).

2. יחידת ה- Datapath:

תוצאות סימולציית tb_Datapath-



סיכום הסימולציה-

על פי תרשים הגלים וה-Testbench, סימולציית ה-Datapath **תקינה לחלוטין** ומציגה פעילות מדויקת של הבאס (Shared BUS), האוגרים ויחידת ה-ALU.

- **טעינת הזיכרון:** בשלב הראשוני, זיכרון ההוראות (ITCM) נטען בהצלחה ב-4 פקודות הבדיקה בצורה טורית ומחזורית.
- **ביצוע פקודות ה-MOV:** המעבד מבצע פענוח והרחבת סימן (Sign Extension) בצורה נכונה, כאשר הערך 0xFF מורחב ל-0xFFFF ונשמר באוגר היעד.
- **פעולות ה-ALU והדגלים:** יחידת ה-ALU מציגה תוצאות אריתמטיות ולוגיות מדויקות. הדגלים (Flags) עולים בזמן ומגיבים בצורה נכונה: פקודת ה-ADD יוצרת גלישה ומעלה את דגל הנשא (C=1) והאפס (Z=1), בעוד שפקודת ה-XOR מפעילה את דגל השלילי (N=1).

ניתוח הסימולציה לפי נקודות זמן-

1. <u>טעינת זיכרון ההוראות - ITCM Loading</u> (סביב 20,000ps−55,000ps)

האות TBactive נמצא בגבוה ('1') ומעביר את השליטה לטסטבנץ'. בכל מחזור שעון (Clock), האות ITCM_tb_wr עולה והערכים של 4 הפקודות נכתבים בזה אחר זה לתוך ה-ITCM לפי הכתובות 000000 עד 000011.

2. <u>העברת השליטה למעבד</u> (סביב 55,000ps)
האות TBactive יורד ל-'0' (Falling edge). ה-Testbench מנתק את הגישה הישירה שלו, וה-Datapath לוקח שליטה מלאה על הבאז המרכזי לקראת תחילת ריצת הפקודות.

3. <u>מחזור FETCH של פקודת MOV הראשונה</u> (סביב 60,000ps)

האותות IRin ו-PCin עולים יחד ל-'1'. הפקודה הראשונה (MOV R1, #0xFF) נשמרת בתוך אוגר ההוראות (IR), וה-PC מקודם ב-1.

4. <u>ביצוע פקודת MOV R1</u> (סביב 70,000ps)

האותות Imm1_in ו-RFin עולים ל-'1'. הערך המיידי 0xFF עובר הרחבת סימן ל-16 ביט, והתוצאה 0xFFFF נכתבת ישירות לתוך אוגר R1 ב-Register File - בחירה לפי "10" = RFaddr_wr_sel

5. <u>ביצוע פקודת MOV R2</u> (סביב 90,000ps)

לאחר מחזור FETCH נוסף, האותות Imm1_in ו-RFin עולים שוב. הערך המיידי 0x0001 נשפך אל הבאס ונכתב לתוך אוגר R2.

6. <u>פענוח פקודת ה-ADD</u> (סביב 100,000ps)

מבוצע מחזור FETCH לפקודה השלישית. מפענח הפקודה (OPC Decoder) מזהה שמדובר בפקודת חיבור, והאות add עולה ל-'1' (Rising edge) כסיגנל סטטוס לקונטרולר.

7. <u>שלב ראשון בחיבור - ALU - ADD R1</u> (סביב 110,000ps)

האותות RFout ו-Ain עולים ל-'1', כאשר בחירת כתובת הקריאה היא "01" = RFaddr_rd_sel (כלומר אוגר R1). הערך 0xFFFF נקרא מה-RF ונכנס לאוגר הכניסה A_reg של ה-ALU.

8. <u>ביצוע החיבור ב- ADD - R2</u> (סביב 120,000ps)

האות ALUFN מוגדר כ-"0000" (פעולת ADD). האותות RFout ו-Cin עולים, והכתובת משתנה ל-"00" (אוגר R2). ה-ALU מחשב-

- 0xFFFF + 0x0001 = 0x0000
- התוצאה ננעלת באוגר C_reg. בעקבות הגלישה והתוצאה, דגלי הסטטוס ננעלים על: Zflag = 1, (תוצאה אפס) ו-Cflag = 1 (קיים נשא)

9. כתיבת תוצאת החיבור - ADD R3 (סביב 130,000ps)

האותות RFin ו-Cout עולים ל-'1', וכתובת הכתיבה נקבעת ל-ra = R0 (לפי "10" = RFaddr_wr_sel). התוצאה 0x0000 נכתבת חזרה ל-Register File.

10. פענוח פקודת ה-XOR (סביב 145,000ps)

מבוצע FETCH לפקודה הרביעית. מפענח הפקודה מזהה את פקודת ה-XOR הלוגית, והאות xorop עולה ל-'1' (Rising edge).

11. בחירת פעולת XOR ב-ALU (סביב 160,000ps)

מה קורה בחומרה: לאחר טעינת אוגר A_reg בערך של R1 , האות ALUFN משתנה ל-"0100" (קוד הפעולה של XOR). האות Cin עולה כדי לנעול את התוצאה הלוגית באוגר C.
החישוב המבוצע:

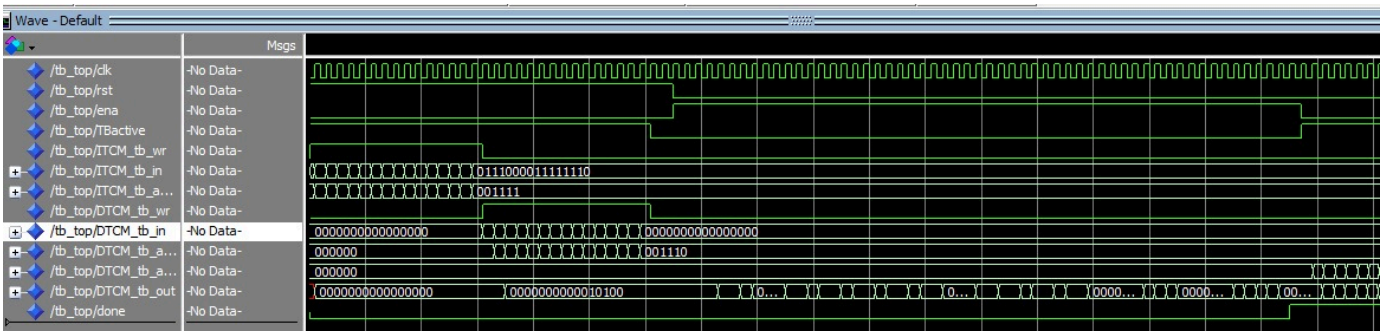
- $0xFFFF \oplus 0x0001 = 0xFFFE$
- בגלל שהביט העליון (MSB) הוא '1', דגל ה-negative מופעל: Nflag = '1'

12. כתיבת תוצאת ה-XOR (סביב 170,000ps)

האותות RFin ו-Cout עולים יחד פעם אחרונה. התוצאה הלוגית 0xFFFE המוצאת מאוגר C נכתבת אל אוגר היעד R3 בתוך ה-Register File.

3. יחידת ה- Top:

תוצאות סימולציית tb_Top-



סיכום הסימולציה-

סימולציית ה-Top Unit מייצגת את הבדיקה הסופית והמקיפה ביותר של הפרויקט, שבה מכונת המצבים (Control) ויחידת הנתונים (Datapath) עובדות יחד בסנכרון מלא. הסימולציה תקינה ומציגה ריצה מלאה של מעבד פונקציונלי.

- **קריאה וכתיבה מקבצים (File I/O):** הטסטבנץ' טוען בהצלחה את הזיכרונות ישירות מתוך קבצי טקסט חיצוניים (ITCMinit.txt ו-DTCMinit.txt) , ובסיום פורק את תוצאות הריצה לקובץ פלט.
- **עבודה עצמאית של ה-CPU:** בניגוד לסימולציית ה-Datapath שבה הסיגנלים נדחפו ידנית, כאן המעבד רץ בצורה אוטומטית לחלוטין (ה-FSM מנווט את הבאס ואת פתיחת השערים בכל מחזור שעון).
- **סנכרון תהליכים:** שלבי הריצה מחולקים בצורה כרונולוגית ברורה ל-4 פאזות מוגדרות.

ניתוח הסימולציה לפי נקודות זמן-

1. <u>טעינת זיכרון ההוראות - ITCM Loading</u> (סביב 30,000os - 0ps)

האות TBactive עולה ל-'1' כדי לנתק את המעבד מהבאס הפנימי. האות ITCM_tb_wr מופעל, והטסטבנץ' קורא שורה אחר שורה מתוך הקובץ ITCMinit.txt וטוען את פקודות המעבד לתוך זיכרון התוכנית.

2. <u>טעינת זיכרון הנתונים - DTCM Loading</u> (סביב 55,000ps - 30,000ps)

אות הכתיבה של זיכרון ההוראות יורד, ובמקומו עולה אות הכתיבה של זיכרון הנתונים- DTCM_tb_wr <= 1. הטסטבנץ' קורא את הערכים המאותחלים מתוך הקובץ DTCMinit.txt ומזין אותם לתוך ה-DTCM. שלב זה מסתיים בנפילת האות סביב 55,000.

3. <u>הרצת המעבד - CPU Running</u> (החל מ-55,000)

עם ירידת האות TBactive ל-'0', המערכת המשולבת מתחילה לפעול עצמאית. קווי הבקרה של ה-FSM מעבירים דגלים ל-Datapath בדיוק לפי סדר ההוראות שנטען-

1. <u>סביב 95,000 - 60,000</u> (פקודות ה-MOV):
   המעבד מבצע מחזורי FETCH וטוען את פקודות ה-MOV. קווי הפיקוד מפעילים את Imm1_in ו-RFin כדי לאתחל את האוגרים R1 ו-R2 בערכים 0xFFFF ו-0x0001.

2. <u>סביב 140,000ps - 100,000ps</u> (פקודת החיבור - ADD):
   ה-FSM מזהה את פקודת ה-ADD. הוא פותח את שערי ה-Register File לטעינת אוגר המקור ('RFout='1', Ain='1') , מורה ל-ALU לבצע חיבור ("ALUFN="0000") ומפעיל את Cin לנעילת התוצאה. בסיום, התוצאה 0x0000 נכתבת חזרה ל-RF יחד עם הפעלת דגלי הנשא והאפס.

3. <u>סביב 175,000ps - 145,000ps</u> (פקודת ה-XOR לוגי):
   המערכת עוברת אוטומטית לפקודה הבאה. קווי הבקרה משתנים, ה-ALUFN מנותב לקוד

"0100" (פעולת XOR) , והתוצאה הלוגית 0xFFFE מחושבת ונשמרת באוגר היעד R3.

4. <u>נקודת סיום הריצה</u> (סביב 180,000 ואילך):
המעבד מגיע לפקודת הסיום הכלולה בתוכנית. מכונת המצבים עוברת למצב S_DONE , ובתגובה מוציאה אות 1' = done' יציב. האות מסמן לטסטבנץ' שהריצה הסתיימה בהצלחה וה-FSM נשאר במצב של לולאה אינסופית.

4. <u>העברת זיכרון הנתונים - DTCM Result Dump</u> (לאחר קבלת done=1)

מיד עם זיהוי הסיגנל done='1', הטסטבנץ' מוריד את אות ה-Enable של המעבד ('0' => ena) ולוקח בחזרה את השליטה על הבאס ('1' => TBactive).

יצירת קובץ הפלט: הטסטבנץ' רץ בלולאה על פני כל כתובות זיכרון הנתונים-(DTCM_tb_addr_out), דוגם את הערכים הסופיים שהשתנו במהלך ריצת התוכנית ומדפיס אותם בצורה מסודרת לתוך קובץ הפלט החיצוני DTCMcontent1.txt לצורך בדיקת נכונות (QA).

על מנת לעבור לפי מודל ה-golden model, הרצנו את ארבעת התרגילים שקיבלנו במשימה על המעבד שלנו, וקיבלנו תוצאות זהות לתוצאות צפויות שקיבלנו כרפרנס, וכך וידאנו שהמעבד שלנו עובד כראוי. לאחר מכן, כתבנו את תרגילים 5 ו-6 והעברנו אותם אמולציה לקוד בינארי שאותו הכנסנו למעבד שלנו, גם הקוד של תרגילים 5 ו-6 הניב תוצאות זהות לתוצאות שקיבלנו ברפרנס, הקוד והתוצאות מצורפות למשימה בהגשה.